

MixBytes()

Enso LayerZero Receiver Security Audit Report

AUGUST 19, 2025

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	3
1.4 Security Assessment Methodology	5
1.5 Risk Classification	7
1.6 Summary of Findings	8
2. Findings Report	9
2.1 Critical	9
2.2 High	9
2.3 Medium	9
M-1 Unrestricted Shortcut Griefing via Low-Gas Calls	9
2.4 Low	10
L-1 Function execute() Can Be Declared as external	10
L-2 Missing Zero Address Validation	11
L-3 Lack of Events for Important Variable Setters and Fund Transfers	12
L-4 Hard-coded reserveGas Threshold Is Inflexible	13
L-5 Inability to Specify Balance in sweep()	14
3. About MixBytes	15

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The `LayerZeroReceiver` contract serves as a `LayerZero` V2 composer that implements the `ILayerZeroComposer` interface to handle `lzCompose` messages from endpoints. It receives bridged tokens and then attempts to execute specified operations through the `EnsoRouter`.

The audit was conducted over 1 day by 3 auditors, involving an in-depth manual code review and automated analysis within the scope.

During the audit, in addition to verifying standard attack vectors and our internal checklist, we conducted an in-depth review of the following areas:

- **Storage of User Funds.** The `LayerZeroReceiver` contract holds user funds for a period of time. We paid special attention to verifying that these funds cannot be stolen during this period.
- **Unauthorized Function Calls.** The project's standard functionality is to accept calls from the LayerZero. We confirmed that a malicious actor cannot make an unauthorized call – impersonating LayerZero – to steal funds belonging to other users or disrupt system operation.
- **Use of Malicious Assets.** We examined whether an attack could be mounted by creating and forwarding a specially crafted asset through the bridge.
- **Self-Call Authorization.** The contract uses Solidity's try/catch mechanism by calling its own `execute()` function. We verified that the `execute()` function is properly protected against calls from other contracts, which ensures only the contract itself can call this function.
- **Error Handling.** We assessed how the contract handles errors in the Enso Router (revert, panic, out of gas), and checked for the presence of retry logic and an emergency mechanism to recover any stuck tokens.
- **Shortcut Execution and Gas Handling**
We have verified the protocol's handling of `lzCompose()` calls, including behavior under constrained gas conditions, to ensure shortcut execution cannot be bypassed or grieved with low-gas transactions and that fallback mechanisms work as expected.

• Native Asset Handling Across Networks

We have verified the protocol's use of `address(0)` as the native asset sentinel and checked for compatibility on networks where the native asset may be represented by a nonzero predeploy contract or alternative standard.

• Replay and Uniqueness Protection

We have verified that GUIDs are processed only once, ensuring messages cannot be replayed or cause double processing.

• Message Integrity and Source Validation.

We confirmed that the message cannot be modified during transmission, as any alteration would cause the `lzCompose()` method to revert due to LayerZero's message hash validation.

Our review revealed no critical vulnerabilities. However, we identified several areas where minor improvements could enhance security, transparency, and operational efficiency. These findings are detailed in the **Findings Report** below.

1.3 Project Overview

Summary

Title	Description
Client Name	Enso Build
Project Name	LayerZero Receiver
Type	Solidity
Platform	EVM
Timeline	15.07.2025 – 21.07.2025

Scope of Audit

File	Link
src/bridge/LayerZeroReceiver.sol	LayerZeroReceiver.sol

Versions Log

Date	Commit Hash	Note
15.07.2025	6531be40c8beb80e568b4ce4ae3ade3d63342440	Initial Commit
17.07.2025	a03f33db35d664330d3b5d3a48a5f70a7860d973	Re-audit Commit
21.07.2025	7914f787be9968e32f93ee641b84bb088f8a690b	2nd Re-audit Commit

Mainnet Deployments

The deployment verification will be conducted later after the full deployment of the protocol into the mainnet.

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	Project Architecture Review: • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: • Cross-check findings among auditors • Discuss identified issues • Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	Bug Fixing & Re-Audit: • The client addresses the identified issues and provides feedback • Auditors verify the fixes and update their statuses with supporting evidence • A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	Final Code Verification & Public Audit Report: • Verify the final code version against recommendations and their statuses • Check deployed contracts for correct initialization parameters • Confirm that the deployed code matches the audited version • Issue a public audit report, published on our official GitHub repository • Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	1
Low	5

Findings Statuses

ID	Finding	Severity	Status
M-1	Unrestricted Shortcut Griefing via Low-Gas Calls	Medium	Fixed
L-1	Function <code>execute()</code> Can Be Declared as <code>external</code>	Low	Fixed
L-2	Missing Zero Address Validation	Low	Fixed
L-3	Lack of Events for Important Variable Setters and Fund Transfers	Low	Fixed
L-4	Hard-coded <code>reserveGas</code> Threshold Is Inflexible	Low	Fixed
L-5	Inability to Specify Balance in <code>sweep()</code>	Low	Fixed

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	Unrestricted Shortcut Griefing via Low-Gas Calls		
Severity	Medium	Status	Fixed in a03f33db

Description

The `lzCompose()` function is externally callable by the `endpoint`, which does not restrict who can invoke it. As a result, any user can call `lzCompose()` with insufficient gas, causing the contract to consistently bypass shortcut execution and fall back to a simple token transfer. This enables griefing attacks where the intended shortcut logic is never executed, even if the sender expects it. The attacker does not need special privileges or to pass any validation, the check only considers whether `gasleft()` is below the set threshold at call time.

While this does not directly result in fund loss, it disrupts protocol guarantees and user experience, making it a **Medium** severity issue.

Recommendation

We recommend transferring the tokens only if the `this.execute()` function fails. Additionally, allow the sender to specify the minimum gas required for executing the shortcut in the message data, thereby providing guarantees that the shortcut won't revert due to insufficient gas.

Client's Commentary

fixed: a03f33db35d664330d3b5d3a48a5f70a7860d973

2.4 Low

L-1	Function <code>execute()</code> Can Be Declared as <code>external</code>		
Severity	Low	Status	Fixed in a03f33db

Description

The `execute()` function is declared as `public`, even though it is only ever called from external callers. Public functions copy their `calldata` into memory, which incurs additional gas overhead compared to `external` functions that read `calldata` directly.

```
function execute(
    address token,
    uint256 amount,
    bytes calldata data,
    uint256 value
) public
```

Recommendation

We recommend declaring this function as `external` to reduce gas consumption.

Client's Commentary:

fixed: a6ecc41691ebb627b3856d39e7418a06bd36b30c

L-2	Missing Zero Address Validation		
Severity	Low	Status	Fixed in a03f33db

Description

The `endpoint` and `router` parameters are not checked for the zero address in the constructor. This could lead to contract deployment with invalid addresses, making the contract non-functional and potentially causing all cross-chain operations to fail.

Recommendation

We recommend adding zero address checks for these parameters in the constructor to prevent deployment with invalid addresses and ensure proper contract initialization.

Client's Commentary:

fixed: a6ecc41691ebb627b3856d39e7418a06bd36b30c

L-3	Lack of Events for Important Variable Setters and Fund Transfers		
Severity	Low	Status	Fixed in 7914f787

Description

The following functions do not emit events when modifying critical state variables:

- `setOFTs()`
- `removeOFTs()`
- `setRegistrar()`
- `removeRegistrar()`

Additionally, the `sweep()` function performs a fund transfer from the contract to the owner without emitting any event, making it difficult to track the movement of assets from the contract on-chain.

This lack of event emission reduces transparency, making it harder to monitor administrative actions or track fund flows off-chain.

Recommendation

We recommend emitting events for the above functions.

Client's Commentary:

Client: fixed - a6ecc41691ebb627b3856d39e7418a06bd36b30c

MixBytes: The issue has been marked as fixed, however, the event for the `sweep()` function has not been implemented.

Client: fixed sweep event - 7914f787be9968e32f93ee641b84bb088f8a690b

L-4	Hard-coded <code>reserveGas</code> Threshold Is Inflexible		
Severity	Low	Status	Fixed in a03f33db

Description

`reserveGas` is declared `immutable` and fixed in the constructor. `lzCompose()` depends on this threshold to decide whether to execute the shortcut or fall back to a direct transfer. Future network upgrades with opcode repricing could render the constant either too low (triggering needless `InsufficientGas` events) or too high (causing avoidable shortcut failures). Since this issue impacts reliability and not security directly, it is categorized as a **Low** severity.

Recommendation

We recommend replacing `reserveGas` with a mutable state variable and exposing an `onlyOwner` setter that emits an event on updates.

Client's Commentary

fixed: a6ecc41691ebb627b3856d39e7418a06bd36b30c

L-5	Inability to Specify Balance in <code>sweep()</code>		
Severity	Low	Status	Fixed in a03f33db

Description

The `sweep()` function currently withdraws the entire balance of each specified token from the contract. This can lead to issues if the contract's balance at any given time includes both "extraneous" tokens (e.g., those tied up in a permanently stuck transaction) and "legitimate" tokens (i.e., those temporarily held as part of the bridge's normal operation). Although such a timing overlap is unlikely, if it does occur, invoking sweep will unintentionally withdraw user-owned tokens. The risk is compounded by the fact that these deposits and sweeps can happen in parallel: when the sweep transaction is submitted, there may be no user tokens in the contract, but by the time it executes, they could have arrived.

Recommendation

We recommend adding an amount parameter to the `sweep()` function's signature to allow the caller to specify exactly how many tokens should be withdrawn.

Client's Commentary

fixed: a03f33db35d664330d3b5d3a48a5f70a7860d973

3. About MixBytes

MixBytes is a leading provider of smart contract audit and research services, helping blockchain projects enhance security and reliability. Since its inception, MixBytes has been committed to safeguarding the Web3 ecosystem by delivering rigorous security assessments and cutting-edge research tailored to DeFi projects.

Our team comprises highly skilled engineers, security experts, and blockchain researchers with deep expertise in formal verification, smart contract auditing, and protocol research. With proven experience in Web3, MixBytes combines in-depth technical knowledge with a proactive security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by top-tier blockchain projects like Lido, Aave, Curve, and others, MixBytes has successfully audited and secured billions in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems.
- **Innovative Research:** Our team actively contributes to blockchain security research, sharing knowledge with the community.

Our Services

- **Smart Contract Audits:** A meticulous security assessment of DeFi protocols to prevent vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical research and security modeling for Web3 projects.
- **Custom Security Solutions:** Tailored security frameworks for complex decentralized applications and blockchain ecosystems.

MixBytes is dedicated to securing the future of blockchain technology by delivering unparalleled security expertise and research-driven solutions. Whether you are launching a DeFi protocol or developing an innovative dApp, we are your trusted security partner.

Contact Information

-  <https://mixbytes.io/>
-  https://github.com/mixbytes/audits_public
-  hello@mixbytes.io
-  <https://x.com/mixbytes>